15

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

프로바이더와 서비스

제2부 NestJS 핵심 · 4장 구성 요소

좌석 규칙을 둘 곳, 그 객체를 만드는 쪽



© 한상훈 NestJS 백엔드 마스터

이번에 만드는 좌석 기능

→ 좌석 열기: 회차마다 A구역 1~3열, 열마다 1~20번, 60석

→ 좌석 붙잡기: 예매 흐름의 두 번째 단계 (좌석 고르기 → 붙잡기 → 결제 → 확정 → 알림)

→ 현장 판매: 공연장 매표소 창구의 판매

→ 공통 규칙: 판매 가능한 좌석만 붙잡기·판매

© 한상훈 NestJS 백엔드 마스터

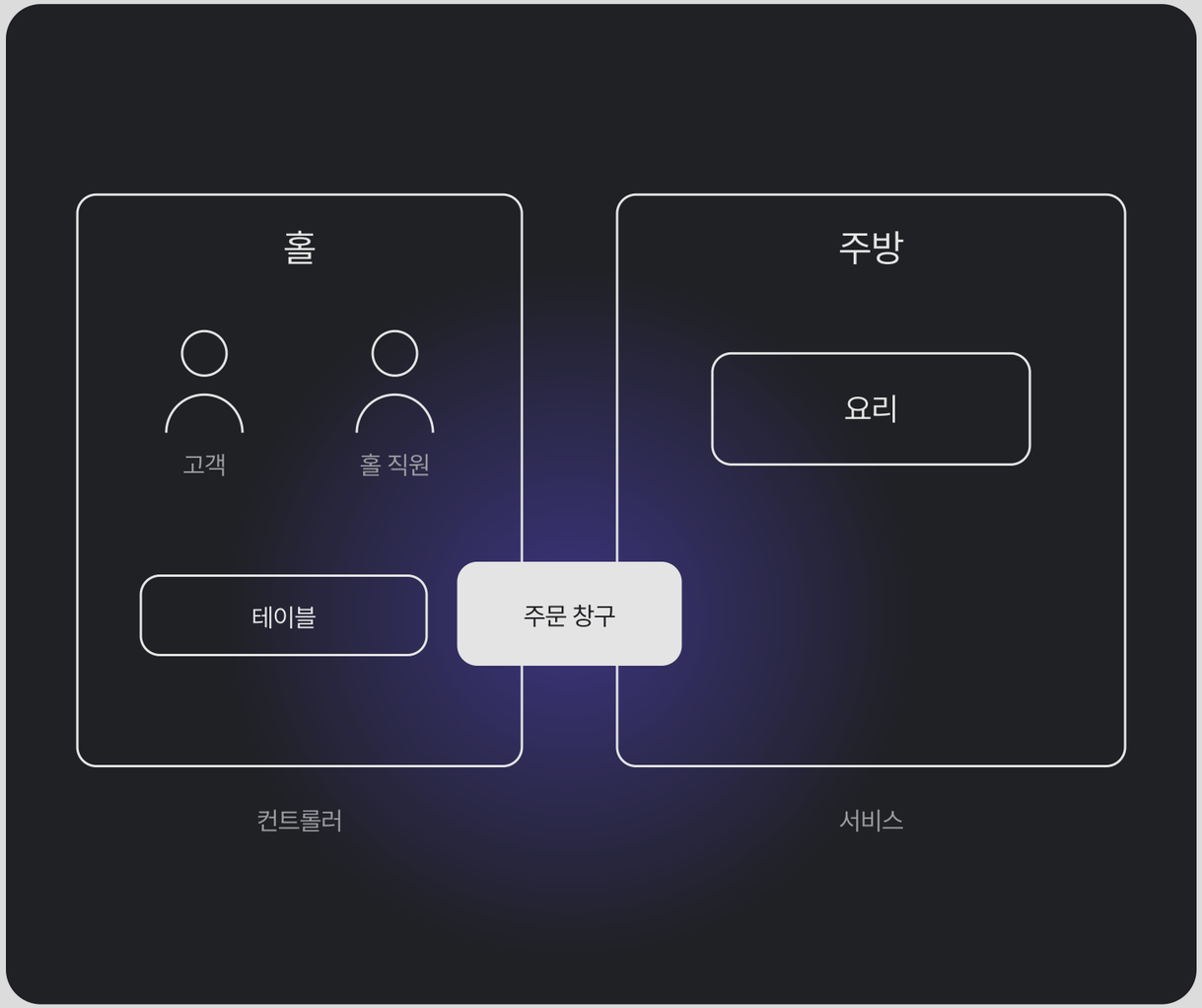
식당 홀과 주방

→ 홀 직원: 주문 접수, 음식 전달

→ 주방: 요리

→ 홀 직원이 요리를 시작하면: 주문이 밀림

→ 컨트롤러 = 홀, 서비스 = 주방



© 한상훈 NestJS 백엔드 마스터

좌석 모듈 만들기

터미널

```
$ nest g module seats --project ticketing
CREATE apps/ticketing/src/seats/seats.module.ts (86 bytes)
UPDATE apps/ticketing/src/app.module.ts (507 bytes)
$ nest g service seats --project ticketing --no-spec
CREATE apps/ticketing/src/seats/seats.service.ts (93 bytes)
UPDATE apps/ticketing/src/seats/seats.module.ts (166 bytes)
$ nest g controller seats --project ticketing --no-spec
CREATE apps/ticketing/src/seats/seats.controller.ts (103 bytes)
UPDATE apps/ticketing/src/seats/seats.module.ts (257 bytes)
```

→ 모듈, 서비스, 컨트롤러를 하나씩: 각 명령이 모듈 파일까지 수정

→ resource를 안 쓴 이유: 좌석은 회차가 열릴 때 한 번에 생성, 이후 상태만 변경 (CRUD 불필요)

→ 공연 모듈의 임시 선점 껍데기(HoldsController, 5만 석 좌석 지도): 삭제, 좌석은 좌석 모듈로

© 한상훈 NestJS 백엔드 마스터

Seat 엔티티

seat.entity.ts

```ts
export type SeatStatus = 'AVAILABLE' | 'HELD' | 'SOLD';

export class Seat {
  id: string;        // 'A-3-15'
  performanceId: number;
  zone: string;
  row: number;
  no: number;
  status: SeatStatus;
}
```

→ AVAILABLE: 판매 가능 / HELD: 붙잡힘 / SOLD: 판매됨

→ id 'A-3-15': 로그·에러 메시지만 보고 좌석 식별

© 한상훈 NestJS 백엔드 마스터

컨트롤러 안의 좌석 규칙

seats.controller.ts

```ts
@Controller('performances/:performanceId/seats')
export class SeatsController {
  private readonly seats = new Map<number, Seat[]>();

  @Post(':seatId/hold')
  hold(@Param('performanceId', ParseIntPipe) performanceId: number, @Param('seatId') seatId: string) {
    const seat = (this.seats.get(performanceId) ?? []).find((s) => s.id === seatId);
    if (!seat) throw new NotFoundException(`좌석 ${seatId}이 없습니다`);
    if (seat.status !== 'AVAILABLE') throw new ConflictException('이미 선택된 좌석입니다');
    seat.status = 'HELD';
    return seat;
  }
}
```

→ 좌석표(Map)와 규칙 전부 컨트롤러 안

→ 409 Conflict: 좌석은 있으나 지금 상태 때문에 불가 (모양 틀림 400, 없음 404)

© 한상훈 NestJS 백엔드 마스터

현장 판매 창구에 복사한 규칙

box-office.controller.ts

```
@Controller('box-office/performances/:performanceId/seats')
export class BoxOfficeController {
  private readonly seats = new Map<number, Seat[]>();

  @Post(':seatId/sell')
  sell(@Param('performanceId', ParseIntPipe) performanceId: number, @Param('seatId') seatId: string) {
    const seat = (this.seats.get(performanceId) ?? []).find((s) => s.id === seatId);
    if (!seat) throw new NotFoundException(`좌석 ${seatId}이 없습니다`);
    if (seat.status === 'SOLD') throw new ConflictException('이미 팔린 좌석입니다');
    seat.status = 'SOLD';
    return seat;
  }
}
```

→ 복사 당시 규칙: SOLD만 거절

© 한상훈 NestJS 백엔드 마스터

두 벌의 규칙

좌석 상태	온라인 붙잡기	창구 판매
AVAILABLE	가능	가능
HELD	409	판매됨 (규칙 누락)
SOLD	409	409

→ 결과: 온라인에서 결제 중인 좌석을 창구가 판매

© 한상훈 NestJS 백엔드 마스터

규칙은 한 곳에

→ 같은 규칙을 쓰는 입구: 온라인, 창구, 관리자 화면, 배치 작업

→ 컨트롤러마다 복사: 고칠 때 전부 찾아야 함

→ 서비스: 규칙과 좌석표를 가진 클래스 하나

→ 컨트롤러: 서비스 메서드 호출만

→ 입구별 테스트: 각자 통과, 두 입구의 차이는 안 보임

© 한상훈 NestJS 백엔드 마스터

SeatsService로 옮긴 규칙

```
export class SeatsService {
  private readonly seats = new Map<number, Seat[]>();

  hold(performanceId: number, seatId: string) {
    const seat = this.takeAvailable(performanceId, seatId);
    seat.status = 'HELD';
    return seat;
  }

  sell(performanceId: number, seatId: string) {
    const seat = this.takeAvailable(performanceId, seatId);
    seat.status = 'SOLD';
    return seat;
  }

  private takeAvailable(performanceId: number, seatId: string) {
    const seat = this.findAll(performanceId).find((s) => s.id === seatId);
    if (!seat) throw new NotFoundException(`좌석 ${seatId}이 없습니다`);
    if (seat.status !== 'AVAILABLE') {
      throw new ConflictException(`${seatLabel(seat.zone, seat.row, seat.no)}은 이미 팔렸거나 선택된 좌석입니다`);
    }
    return seat;
  }
}
```

→ open(좌석 60개 생성), `findAll` 생략 (부록)

→ `seatLabel`: libs/common의 좌석 표기 함수

© 한상훈 NestJS 백엔드 마스터

컨트롤러마다 new

```
export class SeatsController {
  private readonly seatsService = new SeatsService();
  ...
}

export class BoxOfficeController {
  private readonly seatsService = new SeatsService();
  ...
}
```

© 한상훈 NestJS 백엔드 마스터

두 번 찍힌 SeatsService 생성

○ ○ ○ 터미널

```
ConcertsService 생성
SeatsService 생성
SeatsService 생성

$ curl -X POST localhost:4015/performances/1/seats
{"performanceId":1,"count":60}
$ curl -X POST localhost:4015/performances/1/seats/A-3-15/hold
{"id":"A-3-15","performanceId":1,"zone":"A","row":3,"no":15,"status":"HELD"}
$ curl -X POST localhost:4015/performances/1/seats/A-3-15/hold
{"type":"about:blank","title":"Conflict","status":409,
 "detail":"A구역 3열 15번은 이미 팔렸거나 선택된 좌석입니다","instance":"/performances/1/seats/A-3-15/hold"}
$ curl -X POST localhost:4015/box-office/performances/1/seats/A-3-15/sell
{"type":"about:blank","title":"Not Found","status":404,
 "detail":"좌석 A-3-15이 없습니다","instance":"/box-office/performances/1/seats/A-3-15/sell"}
```

→ 생성자 인자 없는 SeatsService: new 가능, 빌드·부팅 정상

© 한상훈 NestJS 백엔드 마스터

좌석표가 두 개

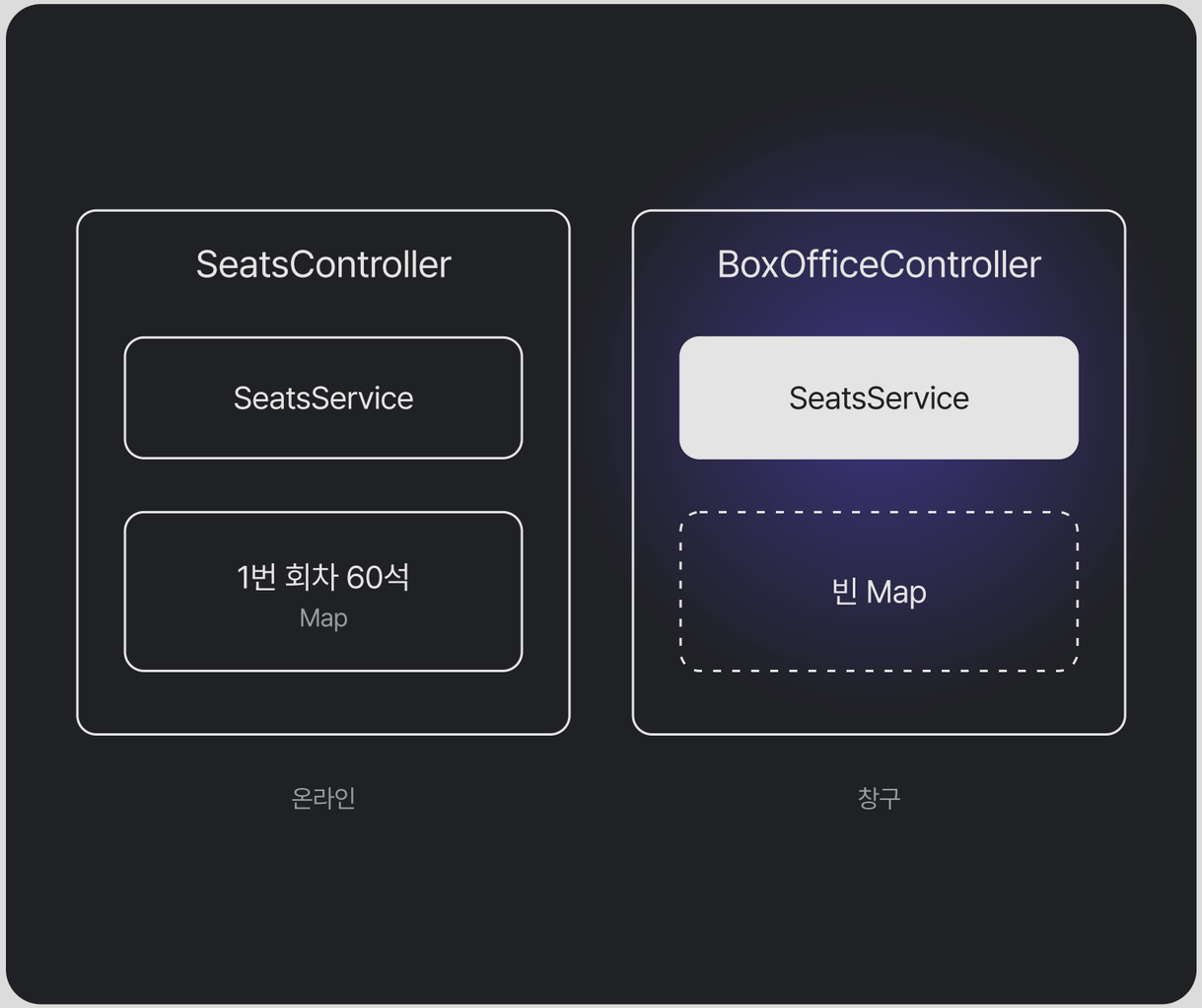
→ 온라인 컨트롤러의 SeatsService: 1번 회차 좌석 60개

→ 창구 컨트롤러의 SeatsService: 빈 Map

→ 규칙은 한 벌, 좌석표는 두 벌

→ 발견: 생성자 로그 두 줄 (로그가 없으면 창구 문의로 발견)

→ 창구에서도 좌석을 따로 열면: 붙잡힌 좌석이 창구에서는 AVAILABLE, 이중 판매



© 한상훈 NestJS 백엔드 마스터

@Injectable과 providers

seats.service.ts

```ts
@Injectable()
export class SeatsService { ... }
```

seats.module.ts

```ts
@Module({
  controllers: [SeatsController, BoxOfficeController],
  providers: [SeatsService],
})
export class SeatsModule {}
```

© 한상훈 NestJS 백엔드 마스터

생성자로 받기

```
export class SeatsController {
  constructor(private readonly seatsService: SeatsService) {}
  ...
}

export class BoxOfficeController {
  constructor(private readonly seatsService: SeatsService) {}
  ...
}
```

→ new 없음: 만드는 일은 Nest

→ 테스트: 진짜 SeatsService 대신 가짜 서비스 주입 가능

© 한상훈 NestJS 백엔드 마스터

하나가 된 좌석표

터미널

```
ConcertsService 생성
SeatsService 생성

$ curl -X POST localhost:4015/performances/1/seats
{"performanceId":1,"count":60}
$ curl -X POST localhost:4015/performances/1/seats/A-3-15/hold
{"id":"A-3-15","performanceId":1,"zone":"A","row":3,"no":15,"status":"HELD"}
$ curl -X POST localhost:4015/box-office/performances/1/seats/A-3-15/sell
{"type":"about:blank","title":"Conflict","status":409,
 "detail":"A구역 3열 15번은 이미 팔렸거나 선택된 좌석입니다","instance":"/box-office/performances/1/seats/A-3-15/sell"}
$ curl -X POST localhost:4015/box-office/performances/1/seats/A-3-16/sell
{"id":"A-3-16","performanceId":1,"zone":"A","row":3,"no":16,"status":"SOLD"}
```

© 한상훈 NestJS 백엔드 마스터

프로바이더와 서비스

→ 프로바이더(provider): Nest가 만들어 보관하다가 생성자에 넣어 주는 객체

→ 등록: 모듈의 providers

→ 서비스(service): 프로바이더 중 업무 규칙을 맡은 클래스 (이름 관례)

→ 주입(injection): 생성자 인자로 넣어 주는 일

→ 기본: 프로세스 안에 하나, 부팅 때 생성

© 한상훈 NestJS 백엔드 마스터

Nest가 부팅 때 하는 일

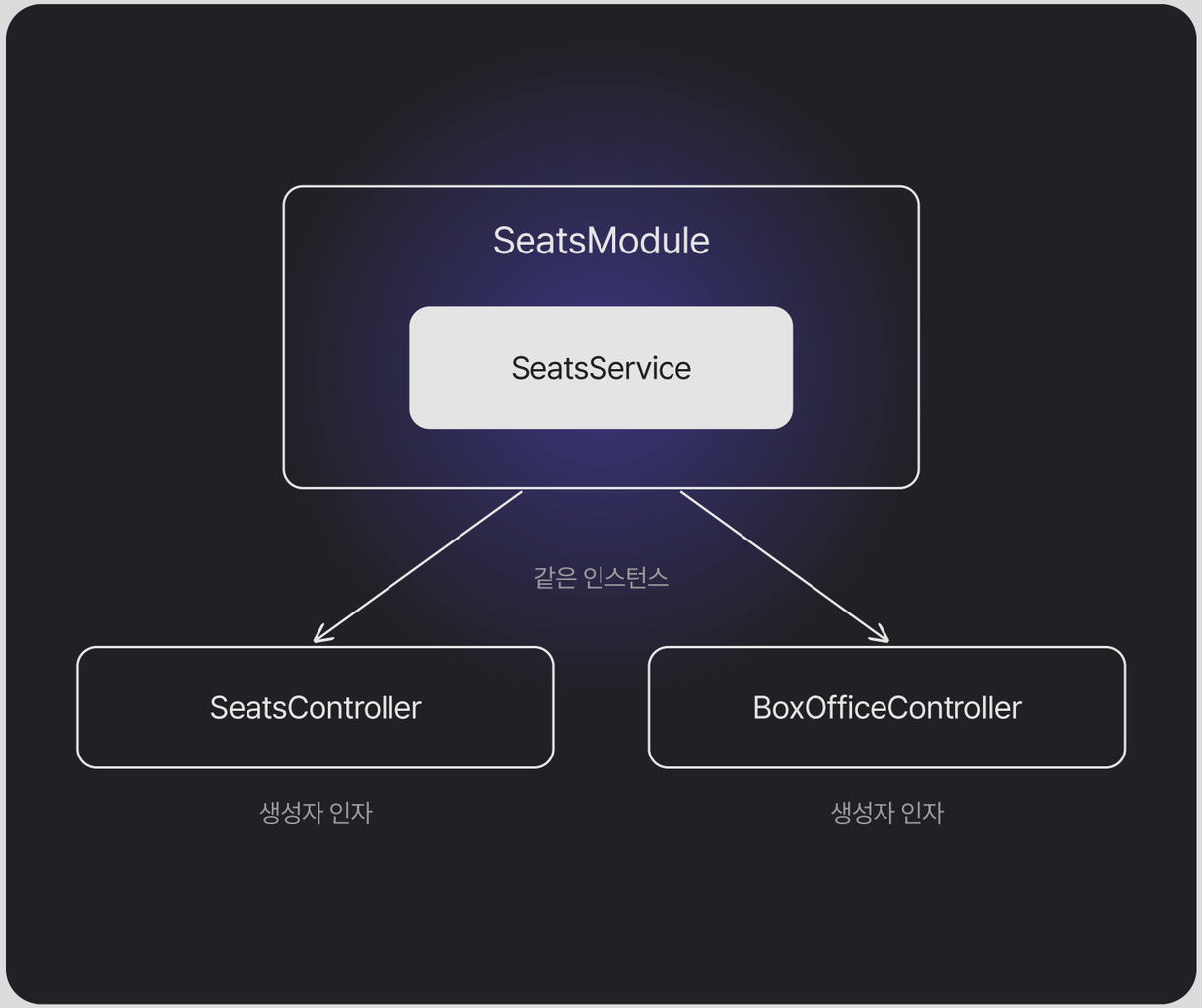
→ SeatsController 생성 차례

→ 생성자 인자 타입 확인: `SeatsService`

→ SeatsModule 안에서 SeatsService 찾기, 없으면 먼저 생성

→ 같은 인스턴스를 BoxOfficeController에도

→ new로 했다면: SeatsService ← PerformancesService ← ConcertsService(공연 API와 같은 인스턴스)까지 직접 맞춰야 함



© 한상훈 NestJS 백엔드 마스터

회차 확인까지

```ts
@Injectable()
export class SeatsService {
  constructor(private readonly performancesService: PerformancesService) {
    console.log('SeatsService 생성');
  }

  open(performanceId: number) {
    this.performancesService.findOne(performanceId);   // 없으면 404
    ...
  }
}
```

○ ○ ○ 터미널

```
$ curl -X POST localhost:4015/performances/7/seats
{"type":"about:blank","title":"Not Found","status":404,"detail":"회차 7번이 없습니다","instance":"/performances/7/seats"}
```

→ SeatsModule `imports: [PerformancesModule]` / PerformancesModule `exports: [PerformancesService]`

© 한상훈 NestJS 백엔드 마스터

데코레이터 한 줄을 지우면

seats.service.ts

```
-@Injectable()
 export class SeatsService {
   constructor(private readonly performancesService: PerformancesService) {
```

→ providers 등록 그대로, 생성자 그대로

© 한상훈 NestJS 백엔드 마스터

뜨는 서버, 통과하는 tsc

○ ○ ○ 터미널

```
$ nest build
Rspack 2.2.9 compiled successfully in 25 ms
$ npx tsc --noEmit -p apps/ticketing/tsconfig.app.json
(출력 없음, 종료 코드 0)
$ node dist/apps/ticketing/main.js
ConcertsService 생성
SeatsService 생성
LOG [InstanceLoader] SeatsModule dependencies initialized +0ms
LOG [NestApplication] Nest application successfully started +0ms
```

© 한상훈 NestJS 백엔드 마스터

첫 요청의 500

터미널

```
$ curl -X POST localhost:4015/performances/1/seats
{"type":"about:blank","title":"Internal Server Error","status":500,
 "detail":"요청을 처리하지 못했습니다.","instance":"/performances/1/seats"}

ERROR [ProblemDetails] TypeError: Cannot read properties of undefined (reading 'findOne')
    at SeatsService.open (.../dist/apps/ticketing/main.js:1276:34)
    at SeatsController.open (.../dist/apps/ticketing/main.js:1155:34)
```

→ `this.performancesService`: undefined

© 한상훈 NestJS 백엔드 마스터

생성자 인자 타입이 남는 조건

→ TypeScript 타입: 빌드 뒤 사라짐

→ 예외: `emitDecoratorMetadata` 설정 + 데코레이터가 붙은 클래스

→ 이때만 생성자 인자 타입 목록을 실행 코드에 기록

→ 기록 없는 클래스: Nest는 인자 없는 생성자로 판단, `new SeatsService()`

→ 확인: `nest g service`로 생성, 새 서비스는 요청 한 번 직접 보내 보기

→ 운영 로그: 생성자로 받은 필드에서 undefined → 데코레이터부터 확인

© 한상훈 NestJS 백엔드 마스터

모든 요청이 보는 Map

→ SeatsService: 프로세스 안에 하나

→ seats Map: 모든 요청, 모든 컨트롤러가 같은 것

→ 좌석표처럼 같이 봐야 하는 값: 원하는 동작

© 한상훈 NestJS 백엔드 마스터

서버가 두 대면

→ 로드 밸런서 뒤 서버 A, 서버 B

→ 서버마다 Node 프로세스, 프로세스마다 SeatsService 하나

→ A에서 붙잡은 A-3-15: B의 Map에서는 AVAILABLE

→ 프로세스 안에서 하나 ≠ 시스템 전체에서 하나



© 한상훈 NestJS 백엔드 마스터

지금 저장소의 한계

항목	지금 (서비스 안 Map)	나중 (DB)
서버 재시작	좌석표 사라짐	유지
서버 두 대	좌석표 두 개	하나
좌석표를 두는 곳	프로세스 메모리 안	프로세스 바깥, 모든 서버가 같이 봄
지금 쓰는 이유	모듈·서비스 구조에 집중	

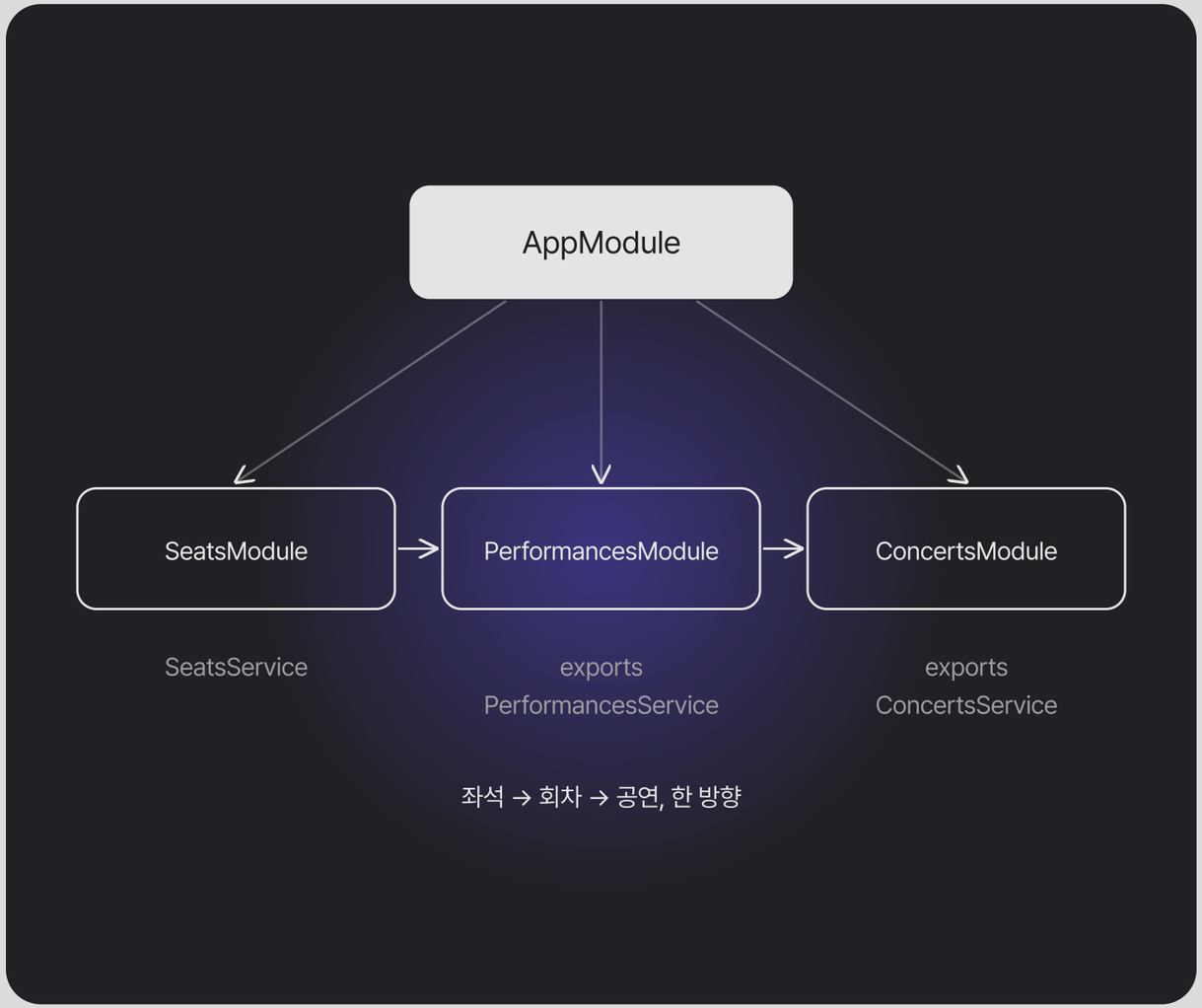
© 한상훈 NestJS 백엔드 마스터

지금의 모듈 그래프

→ AppModule → ConcertsModule, PerformancesModule, SeatsModule

→ SeatsModule → PerformancesModule (exports PerformancesService)

→ PerformancesModule → ConcertsModule (exports ConcertsService)



© 한상훈 NestJS 백엔드 마스터

좌석 API

부팅 로그

```
Mapped {/performances/:performanceId/seats, POST} route
Mapped {/performances/:performanceId/seats, GET} route
Mapped {/performances/:performanceId/seats/:seatId/hold, POST} route
Mapped {/box-office/performances/:performanceId/seats/:seatId/sell, POST} route
```

→ 컨트롤러 둘, 서비스 하나

→ 예: 관리자의 좌석 강제 해제 → 관리자 컨트롤러 + `SeatsService.release`

→ 결제: 결제 서비스 하나, 예매 쪽에서 주입

© 한상훈 NestJS 백엔드 마스터

서비스의 규칙

"규칙은 서비스에 두고, 서비스는 Nest가 하나 만들어서 필요한 곳에 넣어 줍니다."